Flutter PDF Package – Table Creation Documentation

Overview

The **pdf** package in Flutter allows you to generate PDF files dynamically. One of its most usefu creating **tables**, which are commonly used for invoices, reports, lists, and structured data.

Installation

Add the required dependencies to your `pubspec.yaml`:

```
dependencies:
  pdf: ^3.10.8
  printing: ^5.12.0
```

Required Imports

```dart
import 'package:pdf/pdf.dart';
import 'package:pdf/widgets.dart' as pw;
import 'package:printing/printing.dart';
```

Method 1 : Table.fromTextArray (Recommended)

This is the easiest way to create tables.

```dart
Future<void> generatePdf() async {
  final pdf = pw.Document();

  final headers = ['ID', 'Name', 'Age'];
```

```
  final data = [
    ['1', 'John', '25'],
    ['2', 'Alice', '30'],
    ['3', 'Bob', '22'],
  ];

  pdf.addPage(
    pw.MultiPage(
      pageFormat: PdfPageFormat.a4,
      build: (context) => [
        pw.Text(
          'User List',
          style: pw.TextStyle(fontSize: 24),
        ),

        pw.SizedBox(height: 20),

        pw.Table.fromTextArray(
          headers: headers,
          data: data,
          border: pw.TableBorder.all(),
          headerStyle: pw.TextStyle(
            fontWeight: pw.FontWeight.bold,
          ),
          headerDecoration: pw.BoxDecoration(
            color: PdfColors.grey300,
          ),
          cellAlignment: pw.Alignment.centerLeft,
          cellHeight: 30,
        ),
      ],
    ),
  );

  await Printing.layoutPdf(
    onLayout: (format) async => pdf.save(),
  );
}
```

Method 2 : Manual Table Creation (Advanced)

Use this when you need full customization.

```
pw.Table(
  border: pw.TableBorder.all(),
  children: [

    pw.TableRow(
      decoration: pw.BoxDecoration(color: PdfColors.grey300),
      children: [
        pw.Padding(
          padding: pw.EdgeInsets.all(8),
          child: pw.Text('ID'),
        ),
        pw.Padding(
          padding: pw.EdgeInsets.all(8),
          child: pw.Text('Name'),
        ),
        pw.Padding(
          padding: pw.EdgeInsets.all(8),
          child: pw.Text('Age'),
        ),
      ],
    ),

    pw.TableRow(
      children: [
        pw.Padding(
          padding: pw.EdgeInsets.all(8),
          child: pw.Text('1'),
        ),
        pw.Padding(
          padding: pw.EdgeInsets.all(8),
          child: pw.Text('John'),
        ),
        pw.Padding(
          padding: pw.EdgeInsets.all(8),
          child: pw.Text('25'),
        ),
      ],
    ),
  ],
)
```

Dynamic Table from Model

```
class User {
  final String id;
  final String name;
  final int age;

  User(this.id, this.name, this.age);
}

final users = [
  User('1', 'John', 25),
  User('2', 'Alice', 30),
  User('3', 'Bob', 22),
];

final tableData = users.map((user) {
  return [
    user.id,
    user.name,
    user.age.toString(),
  ];
}).toList();
```

Use it like this:

```
pw.Table.fromTextArray(
  headers: ['ID', 'Name', 'Age'],
  data: tableData,
);
```

Styling Options

Border

```
border: pw.TableBorder.all()
```

Header Style

```
headerStyle: pw.TextStyle(
  fontWeight: pw.FontWeight.bold,
  fontSize: 14,
)
```

Header Background

```
headerDecoration: pw.BoxDecoration(
  color: PdfColors.grey300,
)
```

Alignment

```
cellAlignment: pw.Alignment.center
```

Best Practices

- Use `Table.fromTextArray` for simple tables
- Use `MultiPage` for large data
- Convert models to `List<List<String>>`
- Keep table layouts clean and readable

Summary

Feature	Recommended Method
Simple table	Table.fromTextArray
Custom layout	Table
Dynamic data	Convert List → Table
Multi-page support	MultiPage

You can now edit this documentation directly on this page.